

Sistema de Gestão para Oficina Mecânica

Projeto de Banco de Dados

Universidade Federal de Alagoas — UFAL | Campus Arapiraca

Disciplina: BANCO DE DADOS

Docente: Prof. RODOLFO CARNEIRO CAVALCANTE

Discentes: DAVISON GABRIEL MONTEIRO DE FARIAS / EWERTON GABRIEL OLIVEIRA CAVALCANTE

Arapiraca — AL | Junho / 2026

1. Introdução e Justificativa

Este projeto descreve a modelagem e implementação de um banco de dados relacional para gerenciar as operações de uma oficina mecânica de pequeno porte. O sistema cobre o ciclo completo das operações: cadastro de clientes e veículos, abertura e gestão de Ordens de Serviço (OS), controle de estoque de peças, rateio de comissões entre mecânicos, faturamento e acionamento de garantias pós-serviço.

Justificativa para o uso de Banco de Dados Relacional:

O ecossistema de uma oficina exige forte integridade transacional. É imperativo que a baixa de estoque de uma peça, o faturamento financeiro e o registro de serviços no veículo ocorram de forma atômica e consistente. O modelo relacional garante, através de chaves primárias, chaves estrangeiras, restrições de unicidade e políticas de deleção, que não existam dados órfãos — como pagamentos sem Ordem de Serviço ou deduções de estoque sem peça cadastrada. Além disso, a natureza altamente relacional dos dados (clientes veículos OS serviços mecânicos pagamentos) torna o modelo relacional a escolha natural e superior a abordagens não estruturadas.

2. Descrição do Mini-Mundo

2.1. Perfil dos Usuários

- **Atendentes:** Responsáveis pela interface com o cliente, abertura de Ordens de Serviço, registro de vendas de balcão e geração de solicitações de compra de peças.
- **Mecânicos:** Executam os serviços, consomem as peças requisitadas e dividem comissões através do sistema de rateio percentual.
- **Gerência:** Consulta relatórios de faturamento, monitora giro de estoque e acompanha o status das OS em aberto.

2.2. Dados Armazenados

O sistema armazena e gerencia as seguintes categorias de informação:

- **Pessoas:** Dados cadastrais unificados de clientes e funcionários (atendentes e mecânicos), com suporte a múltiplos telefones.
- **Localização:** Estados e cidades normalizados para eliminar redundância de endereço.
- **Veículos:** Frota dos clientes com identificação por placa, incluindo marca, modelo, ano, cor e tipo de combustível.
- **Catálogo de Peças:** Estoque com controle de quantidade e valor unitário.
- **Catálogo de Serviços:** Tabela de preços com tempo estimado e parâmetros de garantia (prazo em dias e limite em quilometragem).
- **Ordens de Serviço:** Ciclo completo desde orçamento até finalização, com categoria (manutenção ou venda balcão), prioridade e datas de abertura, estimativa e fechamento.
- **Itens da OS:** Serviços executados e peças consumidas por OS, com snapshot de preço no momento da venda.
- **Rateio:** Distribuição percentual de comissões entre mecânicos por item de serviço.
- **Pagamentos:** Registro de recebimentos com tipo, valor e status.
- **Garantias:** Controle pós-venda com prazo e quilometragem, vinculado ao item específico executado.
- **Solicitações de Compra:** Reposição de estoque com rastreabilidade do atendente responsável.

2.3. Regras de Negócio e Restrições de Integridade

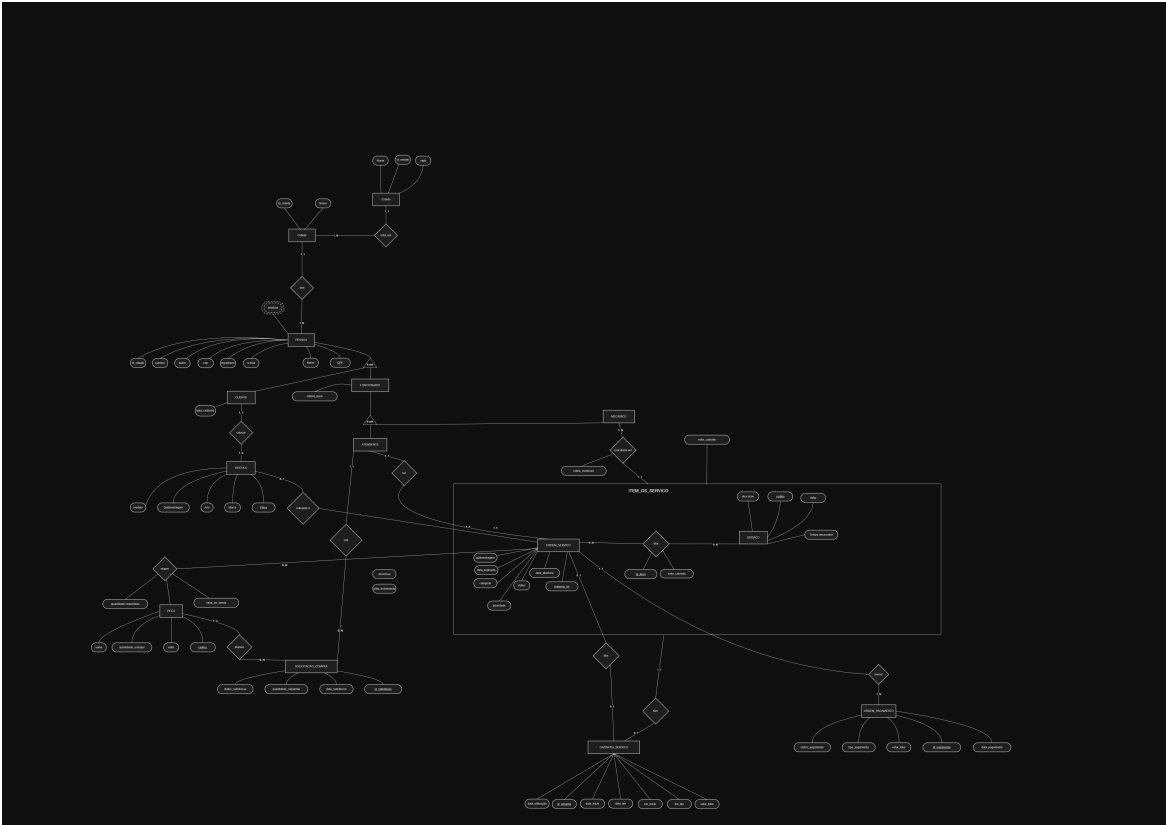
Regra	Descrição
Especialização de Pessoa	Uma Pessoa pode ser Cliente ou Funcionário. O Funcionário subdivide-se exclusivamente em Mecânico ou Atendente.
Validação de OS	OS do tipo <code>Manutencao_Veicular</code> exigem placa de veículo e quilometragem. OS do tipo <code>Venda_Balcao</code> não podem ter placa

	nem quilometragem.
Garantia controlada	Garantias são geradas apenas para serviços de manutenção veicular em OS finalizadas. A dimensão de quilometragem não se aplica a vendas de balcão.
Rateio completo	A soma dos percentuais de rateio entre mecânicos em um mesmo item de serviço deve totalizar exatamente 100%.
Snapshot de preço	O valor cobrado por serviço ou peça é registrado no momento da OS, independentemente de alterações futuras no catálogo.
Valor total derivado	O valor total da OS não é armazenado fisicamente — é calculado em tempo real pela view <code>vw_faturamento_os</code> para evitar anomalias de atualização.
Datas consistentes	A data estimada de entrega não pode ser anterior à data de abertura. A data de fechamento, quando preenchida, também não pode ser anterior à abertura.
Salário positivo	O salário base de um funcionário deve ser maior que zero.
Estoque não negativo	A quantidade em estoque de uma peça não pode ser negativa.
Quilometragem não negativa	A quilometragem registrada na OS deve ser maior ou igual a zero.

2.4. Casos de Uso Principais

1. **Manutenção preventiva completa:** Abertura de OS de manutenção, alocação de serviços com divisão de execução entre múltiplos mecânicos, consumo de peças do estoque e geração automática de garantias ao finalizar.
 2. **Venda de balcão:** Registro de venda direta de peças sem vínculo a veículo, simplificando o processo sem criar tabelas extras.
 3. **Retorno em garantia:** Reabertura de OS para refação de serviço coberto por garantia, com rastreabilidade da OS original que gerou a cobertura.
 4. **Controle de estoque:** Monitoramento de peças com solicitação de compra manual ou automática (via trigger) quando o estoque cai abaixo do mínimo.
 5. **Faturamento em tempo real:** Consulta do valor total de qualquer OS diretamente pela view, consolidando serviços e peças dinamicamente.
-

3. Modelagem Conceitual



3.1. Entidades e Atributos

Entidade	Atributos relevantes
Pessoa	CPF (simples, PK), nome (simples), email (simples), endereço (composto: logradouro, número, bairro, CEP, cidade), telefones (multivalorado)
Funcionário	salário_base (simples)
Atendente	— (especialização de Funcionário)
Mecânico	— (especialização de Funcionário)
Cliente	data_cadastro (simples, derivado de NOW())
Veículo	placa (simples, PK), marca, modelo, ano, cor, tipo_combustivel
Peça	codigo_peca (PK), nome, quantidade_estoque, valor_peca
Serviço	codigo_servico (PK), descricao, valor, tempo_estimado_horas, garantia_dias, garantia_km

Ordem de Serviço	codigo_os (PK), descricao, categoria, prioridade, status_os, data_abertura, data_estimada, data_fechamento, quilometragem
Item OS Serviço	id_item (PK), valor_cobrado (snapshot)
Ordem Serviço Peça	id_item_peca (PK), quantidade_requisitada, valor_unitario_cobrado (snapshot)
Rateio Mecânico	percentual_rateio
Ordem de Pagamento	id_pagamento (PK), tipo_pagamento, status_pagamento, valor_pago, data_pagamento
Garantia Serviço	id_garantia (PK), data_inicio, data_fim, km_inicio, km_fim, utilizada, data_utilizacao
Solicitação de Compra	id_solicitacao (PK), status_solicitacao, data_solicitacao, quantidade_requerida
Estado	id_estado (PK), sigla, nome
Cidade	id_cidade (PK), nome

3.2. Atributo Derivado

O **valor total da OS** é um atributo derivado — calculado a partir da soma dos valores em `item_os_servico` e `ordem_servico_peca`. Por isso não é armazenado fisicamente na tabela, sendo materializado exclusivamente pela view `vw_faturamento_os`.

3.3. Especialização / Generalização

`Pessoa` é a entidade genérica. A especialização é **disjunta e parcial**: uma Pessoa pode ser Cliente, Funcionário, ou ambos (ex: um funcionário que também é cliente da oficina). O Funcionário se especializa disjuntamente em Atendente ou Mecânico — um funcionário não pode exercer os dois papéis simultaneamente no modelo atual.

A implementação utiliza o padrão **Table Per Type**: cada especialização possui sua própria tabela com chave primária referenciando a tabela pai, evitando colunas nulas e mantendo a 3FN.

3.4. Agregação (Entidade Associativa)

O relacionamento entre `ORDEM_SERVICO` e `SERVICO` foi modelado como uma entidade associativa (`ITEM_OS_SERVICO`). Essa decisão foi necessária porque `MECÂNICO` precisa se relacionar com o serviço prestado **dentro de uma OS específica**, e não com o serviço genérico do catálogo. Sem a agregação, não seria possível no modelo ER ligar uma entidade diretamente a um relacionamento — a entidade associativa resolve isso tornando o relacionamento em uma entidade de pleno direito.

3.5. Cardinalidades

Relacionamento	Cardinalidade	Participação
Pessoa — Telefones	1:N	Parcial (nem toda pessoa tem telefone)
Cliente — Veículo	1:N	Parcial (cliente pode não ter veículo)
Atendente — OS	1:N	Total (toda OS tem um atendente)
OS — Item OS Serviço	1:N	Parcial (OS de balcão pode ter só peças)
OS — Ordem Serviço Peça	1:N	Parcial (OS de serviço puro não usa peças)
Item OS Serviço — Rateio	1:N	Total (todo serviço executado deve ter mecânico)
Item OS Serviço — Garantia	1:1	Parcial (apenas serviços de manutenção finalizada)
OS — Ordem Pagamento	1:N	Parcial (OS em aberto ainda sem pagamento)
Peça — Solicitação Compra	1:N	Parcial

4. Mapeamento Lógico e Normalização

4.1. Mapeamento ER Relacional

Cada entidade forte foi mapeada diretamente para uma tabela com sua chave primária. As entidades fracas e associativas receberam chaves estrangeiras referenciando as

entidades participantes. O atributo multivalorado `telefone` foi mapeado para a tabela separada `mecanica.telefones`.

A hierarquia de especialização foi implementada com **Table Per Type**:

```

pessoa (cpf PK)
  funcionario (cpf_funcionario PK  pessoa.cpf)
    atendente (cpf_atendente PK  funcionario.cpf_funcionario)
    mecanico (cpf_mecanico PK  funcionario.cpf_funcionario)
  cliente (cpf_cliente PK  pessoa.cpf)
```

4.2. Normalização até a 3FN

1FN — Primeira Forma Normal: Todos os atributos são atômicos. O atributo multivalorado `telefone` foi extraído para tabela própria. Não há grupos repetitivos.

2FN — Segunda Forma Normal: Todas as tabelas utilizam chave primária simples (não composta), exceto `rateio_mecanico_servico` cuja PK é `(id_item, cpf_mecanico)`. Todos os atributos não-chave dependem funcionalmente da chave inteira — `percentual_rateio` depende do par (item, mecânico), não de apenas um deles.

3FN — Terceira Forma Normal: Não há dependências transitivas. O caso mais relevante é o endereço: inicialmente `cidade` e `estado` estariam em `pessoa`, criando a dependência transitiva `cpf cep cidade estado`. Isso foi resolvido normalizando `estado` e `cidade` em tabelas próprias, referenciadas por `id_cidade` em `pessoa`.

A decisão mais relevante de 3FN foi a **remoção da coluna `valor_total`** de `ordem_servico`:

- **Problema:** O valor total depende funcionalmente dos itens da OS (`item_os_servico` e `ordem_servico_peca`), não da OS em si. Armazená-lo criaria dependência transitiva e risco de anomalia de atualização.
- **Solução:** O valor é calculado dinamicamente pela view `vw_faturamento_os`, garantindo que o dado seja sempre consistente sem necessidade de manutenção manual.

4.3. Políticas de Deleção (ON DELETE)

Relacionamento	Política	Justificativa
<code>pessoa</code> <code>telefones</code>	CASCADE	Telefone não existe sem pessoa; faz sentido remover junto

pessoa	funcionario	RESTRICT	Não remover pessoa com vínculo empregatício ativo
pessoa	cliente	RESTRICT	Não remover cliente com histórico de OS
funcionario	atendente	RESTRICT	Atendente com OS abertas não pode ser removido
funcionario	mecanico	RESTRICT	Mecânico com rateios registrados não pode ser removido
cliente	veiculo	RESTRICT	Veículo com OS vinculada não pode ser removido
veiculo	ordem_servico	SET NULL	Veículo removido: OS mantida como histórico, placa nulificada
ordem_servico	item_os_servico	CASCADE	Itens não existem sem a OS
ordem_servico	ordem_servico_peca	CASCADE	Idem
ordem_servico	ordem_pagamento	RESTRICT	Pagamento registrado não pode ser apagado
item_os_servico	rateio_mecanico_servico	CASCADE	Rateio não existe sem o item
item_os_servico	garantia_servico	RESTRICT	Garantia ativa não permite remoção do item
peca	ordem_servico_peca	RESTRICT	Peça com histórico de consumo não pode ser removida
peca	solicitacao_compra	RESTRICT	Solicitação ativa não permite remoção da peça

5. Implementação Física (DDL)

O script DDL foi implementado para PostgreSQL, com tipos de dados otimizados, ENUMs para domínios controlados e restrições aplicadas diretamente no banco, garantindo

integridade independentemente da camada de aplicação.

5.1. Tipos Enumerados (ENUMs)

```
CREATE TYPE mecanica.status_solicitacao AS ENUM (  
    'Aberto', 'Em_andamento', 'Finalizado', 'Cancelado'  
);  
CREATE TYPE mecanica.status_pagamento AS ENUM (  
    'Pago', 'Pendente', 'Parcial', 'Estornado'  
);  
CREATE TYPE mecanica.tipo_pagamento AS ENUM (  
    'Pix', 'Dinheiro', 'Cartao_Debito', 'Cartao_Credito', 'Boleto'  
);  
CREATE TYPE mecanica.prioridade AS ENUM (  
    'Baixa', 'Normal', 'Alta', 'Urgente'  
);  
CREATE TYPE mecanica.status_os AS ENUM (  
    'Orcamento', 'Aguardando_Aprovacao', 'Aprovado',  
    'Em_Andamento', 'Aguardando_Peca', 'Finalizado', 'Cancelado'  
);  
CREATE TYPE mecanica.categoria AS ENUM (  
    'Venda_Balcao', 'Manutencao_Veicular'  
);  
CREATE TYPE mecanica.tipo_combustivel AS ENUM (  
    'Gasolina', 'Diesel', 'Flex'  
);
```

5.2. Tabelas Principais

```
CREATE TABLE mecanica.estado (  
    id_estado SERIAL PRIMARY KEY,  
    sigla CHAR(2) NOT NULL UNIQUE,  
    nome VARCHAR(50) NOT NULL UNIQUE  
);  
  
CREATE TABLE mecanica.cidade (  
    id_cidade SERIAL PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    id_estado INTEGER NOT NULL
```

```

        REFERENCES mecanica.estado(id_estado) ON DELETE RESTRICT,
        UNIQUE (nome, id_estado)
    );

CREATE TABLE mecanica.pessoa (
    cpf    VARCHAR(11) PRIMARY KEY,
    nome   VARCHAR(100) NOT NULL,
    email  VARCHAR(254) NOT NULL UNIQUE,
    logradouro VARCHAR(100) NOT NULL,
    numero VARCHAR(10) NOT NULL,
    bairro VARCHAR(50) NOT NULL,
    cep    VARCHAR(8) NOT NULL,
    id_cidade INTEGER NOT NULL
        REFERENCES mecanica.cidade(id_cidade) ON DELETE RESTRICT
);

CREATE TABLE mecanica.telefones (
    id_telefone SERIAL PRIMARY KEY,
    cpf_pessoa VARCHAR(11) NOT NULL
        REFERENCES mecanica.pessoa(cpf) ON DELETE CASCADE,
    telefone VARCHAR(20) NOT NULL,
    UNIQUE (cpf_pessoa, telefone)
);

CREATE TABLE mecanica.funcionario (
    cpf_funcionario VARCHAR(11) PRIMARY KEY
        REFERENCES mecanica.pessoa(cpf) ON DELETE RESTRICT,
    salario_base NUMERIC(10, 2) NOT NULL CHECK (salario_base > 0)
);

CREATE TABLE mecanica.atendente (
    cpf_atendente VARCHAR(11) PRIMARY KEY
        REFERENCES mecanica.funcionario(cpf_funcionario) ON DELETE RESTRICT
);

CREATE TABLE mecanica.mecanico (
    cpf_mecanico VARCHAR(11) PRIMARY KEY
        REFERENCES mecanica.funcionario(cpf_funcionario) ON DELETE RESTRICT
);

```

```
CREATE TABLE mecanica.cliente (  
  cpf_cliente VARCHAR(11) PRIMARY KEY  
  REFERENCES mecanica.pessoa(cpf) ON DELETE RESTRICT,  
  data_cadastro TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

```
CREATE TABLE mecanica.veiculo (  
  placa VARCHAR(10) PRIMARY KEY,  
  marca VARCHAR(40) NOT NULL,  
  ano SMALLINT NOT NULL CHECK (ano >= 1900),  
  cor VARCHAR(20) NOT NULL,  
  tipo_combustivel mecanica.tipo_combustivel NOT NULL,  
  cpf_dono VARCHAR(11) NOT NULL  
  REFERENCES mecanica.cliente(cpf_cliente) ON DELETE RESTRICT,  
  modelo VARCHAR(80) NOT NULL  
);
```

```
CREATE TABLE mecanica.pecas (  
  codigo_pecas SERIAL PRIMARY KEY,  
  nome VARCHAR(80) NOT NULL,  
  quantidade_estoque INT NOT NULL CHECK (quantidade_estoque >= 0),  
  valor_pecas NUMERIC(10, 2) NOT NULL CHECK (valor_pecas > 0)  
);
```

```
CREATE TABLE mecanica.solicitacao_compra (  
  id_solicitacao SERIAL PRIMARY KEY,  
  cpf_atendente VARCHAR(11) NOT NULL  
  REFERENCES mecanica.atendente(cpf_atendente) ON DELETE RESTRICT,  
  status_solicitacao mecanica.status_solicitacao NOT NULL DEFAULT 'Aberto',  
  data_solicitacao DATE DEFAULT CURRENT_DATE,  
  quantidade_requerida SMALLINT NOT NULL CHECK (quantidade_requerida > 0),  
  pecas_solicitadas INTEGER NOT NULL  
  REFERENCES mecanica.pecas(codigo_pecas) ON DELETE RESTRICT  
);
```

```
CREATE TABLE mecanica.servico (  
  codigo_servico SERIAL PRIMARY KEY,  
  descricao TEXT NOT NULL,  
  valor NUMERIC(10, 2) NOT NULL CHECK (valor > 0),  
  tempo_estimado_horas SMALLINT NOT NULL CHECK (tempo_estimado_horas > 0),
```

```

    garantia_dias    SMALLINT    NOT NULL DEFAULT 90 CHECK (garantia_dias >= 0),
    garantia_km      INT         NOT NULL DEFAULT 3000 CHECK (garantia_km >= 0)
);

```

```

CREATE TABLE mecanica.ordem_servico (
    codigo_os        SERIAL          PRIMARY KEY,
    descricao        TEXT            NOT NULL,
    data_fechamento DATE,
    data_estimada    DATE            NOT NULL,
    quilometragem    INT             CHECK (quilometragem >= 0),
    categoria        mecanica.categoria NOT NULL,
    prioridade       mecanica.prioridade NOT NULL DEFAULT 'Normal',
    data_abertura    DATE            NOT NULL DEFAULT CURRENT_DATE,
    status_os        mecanica.status_os NOT NULL DEFAULT 'Orcamento',
    cpf_atendente    VARCHAR(11)     NOT NULL
    REFERENCES mecanica.atendente(cpf_atendente) ON DELETE RESTRICT,
    placa_veiculo    VARCHAR(10)
    REFERENCES mecanica.veiculo(placa) ON DELETE SET NULL,
    CONSTRAINT chk_data_estimada
        CHECK (data_estimada >= data_abertura),
    CONSTRAINT chk_data_fechamento
        CHECK (data_fechamento IS NULL OR data_fechamento >= data_abertura),
    CONSTRAINT chk_regra_negocio_os CHECK (
        (categoria = 'Manutencao_Veicular'
         AND placa_veiculo IS NOT NULL
         AND quilometragem IS NOT NULL)
        OR
        (categoria = 'Venda_Balcao'
         AND placa_veiculo IS NULL
         AND quilometragem IS NULL)
    )
);

```

```

CREATE TABLE mecanica.ordem_pagamento (
    id_pagamento    SERIAL          PRIMARY KEY,
    status_pagamento mecanica.status_pagamento DEFAULT 'Pendente',
    tipo_pagamento  mecanica.tipo_pagamento NOT NULL,
    valor_pago       NUMERIC(10, 2)   NOT NULL CHECK (valor_pago > 0),
    data_pagamento  DATE,
    codigo_os        INTEGER          NOT NULL

```

```

REFERENCES mecanica.ordem_servico(codigo_os) ON DELETE RESTRICT,
UNIQUE (codigo_os, tipo_pagamento)
);

CREATE TABLE mecanica.ordem_servico_peca (
  id_item_peca      SERIAL      PRIMARY KEY,
  codigo_os         INTEGER     NOT NULL
    REFERENCES mecanica.ordem_servico(codigo_os) ON DELETE CASCADE,
  codigo_peca       INTEGER     NOT NULL
    REFERENCES mecanica.peca(codigo_peca) ON DELETE RESTRICT,
  quantidade_requisitada SMALLINT NOT NULL CHECK (quantidade_requisitada > 0),
  valor_unitario_cobrado NUMERIC(10, 2) NOT NULL CHECK (valor_unitario_cobrado > 0),
  UNIQUE (codigo_os, codigo_peca)
);

CREATE TABLE mecanica.item_os_servico (
  id_item      SERIAL      PRIMARY KEY,
  codigo_os    INTEGER     NOT NULL
    REFERENCES mecanica.ordem_servico(codigo_os) ON DELETE CASCADE,
  codigo_servico INTEGER     NOT NULL
    REFERENCES mecanica.servico(codigo_servico) ON DELETE RESTRICT,
  valor_cobrado NUMERIC(10, 2) NOT NULL CHECK (valor_cobrado > 0),
  UNIQUE (codigo_os, codigo_servico)
);

CREATE TABLE mecanica.rateio_mecanico_servico (
  id_item      INTEGER     NOT NULL
    REFERENCES mecanica.item_os_servico(id_item) ON DELETE CASCADE,
  cpf_mecanico VARCHAR(11) NOT NULL
    REFERENCES mecanica.mecanico(cpf_mecanico) ON DELETE RESTRICT,
  percentual_rateio NUMERIC(5, 2) NOT NULL
    CHECK (percentual_rateio > 0 AND percentual_rateio <= 100),
  PRIMARY KEY (id_item, cpf_mecanico)
);

CREATE TABLE mecanica.garantia_servico (
  id_garantia  SERIAL      PRIMARY KEY,
  id_item      INTEGER     NOT NULL
    REFERENCES mecanica.item_os_servico(id_item) ON DELETE RESTRICT,
  data_inicio  DATE        NOT NULL,

```

```
data_fim    DATE    NOT NULL CHECK (data_fim > data_inicio),
km_inicio   INT,
km_fim      INT,
utilizada   BOOLEAN NOT NULL DEFAULT FALSE,
data_utilizacao DATE,
os_retorno  INTEGER
            REFERENCES mecanica.ordem_servico(codigo_os) ON DELETE SET NULL,
            CHECK (km_fim IS NULL OR km_fim >= km_inicio)
);
```

6. Lógica de Banco de Dados (View e Procedure)

6.1. View — Faturamento da OS

Substitui a coluna `valor_total` removida da tabela, calculando o faturamento real dinamicamente a partir dos itens de serviço e peças de cada OS.

```
CREATE OR REPLACE VIEW mecanica.vw_faturamento_os AS
SELECT
  os.codigo_os,
  os.descricao,
  os.placa_veiculo,
  os.categoria,
  os.status_os,
  os.data_abertura,
  os.data_fechamento,
  COALESCE((
    SELECT SUM(valor_cobrado)
    FROM mecanica.item_os_servico
    WHERE codigo_os = os.codigo_os
  ), 0)
  +
  COALESCE((
    SELECT SUM(valor_unitario_cobrado * quantidade_requisitada)
    FROM mecanica.ordem_servico_peca
    WHERE codigo_os = os.codigo_os
  ), 0) AS valor_total_fatura
FROM mecanica.ordem_servico os;
```

6.2. Abstração e Proteção de Dados (vw_painel_mecanico)

Para o perfil técnico do mecânico, o acesso direto às tabelas base que contêm informações pessoais de clientes (como CPF, telefone e endereço) ou dados puramente financeiros (como faturamento e custos) foi bloqueado. Em substituição, implementou-se a `vw_painel_mecanico`. Foi utilizado o recurso de INNER JOIN, interligando as tabelas de ordens de serviço, veículos, clientes e pessoas físicas através de suas respectivas chaves estrangeiras.

A view atua em duas frentes de proteção:

- **Filtro Vertical (Colunas):** Exibe apenas os atributos estritamente necessários para a execução física do serviço (dados do veículo, nome do cliente para identificação no pátio, prazo de entrega, prioridade e defeito relatado). Dados confidenciais financeiros ou de identificação civil são ocultados.
- **Filtro Horizontal (Linhas):** Utiliza uma cláusula condicional baseada no status da Ordem de Serviço (`WHERE`), exibindo em tempo real unicamente os veículos com serviços pendentes, em andamento ou aguardando peças na oficina. Isso limpa a fila de trabalho de registros históricos ou orçamentos não aprovados.

Implementação da View Restrita:

```
-- =====
-- VIEW: Painel de Trabalho do Mecânico (Dados Mascarados)
-- =====

CREATE OR REPLACE VIEW mecanica.vw_painel_mecanico AS
SELECT
    os.codigo_os,
    os.prioridade,
    os.status_os,
    os.data_estimada AS prazo_entrega,
    v.placa AS placa_veiculo,
    v.marca,
    v.modelo,
    v.ano,
    p_cliente.nome AS nome_cliente,
    os.descricao AS defeito_relatado
FROM mecanica.ordem_servico os
INNER JOIN mecanica.veiculo v ON os.placa_veiculo = v.placa
INNER JOIN mecanica.cliente c ON v.cpf_dono = c.cpf_cliente
INNER JOIN mecanica.pessoa p_cliente ON c.cpf_cliente = p_cliente.cpf
```

```
-- Filtro de Linhas: Mostra apenas Ordens de Serviço ativas na oficina
WHERE os.status_os IN ('Aprovado', 'Em_Andamento', 'Aguardando_Peca');
```

6.3. Procedure — Finalizar OS

Centraliza a lógica de finalização em um único ponto, validando o estado da OS antes de agir e gerando as garantias automaticamente para serviços de manutenção veicular.

```
CREATE OR REPLACE PROCEDURE mecanica.sp_finalizar_os(p_codigo_os INTEGER)
LANGUAGE plpgsql
AS $$
DECLARE
    v_status mecanica.status_os;
BEGIN
    SELECT status_os INTO v_status
    FROM mecanica.ordem_servico
    WHERE codigo_os = p_codigo_os;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'OS % não encontrada', p_codigo_os;
    END IF;

    IF v_status = 'Finalizado' THEN
        RAISE EXCEPTION 'OS % já está finalizada', p_codigo_os;
    END IF;

    IF v_status = 'Cancelado' THEN
        RAISE EXCEPTION 'OS % está cancelada e não pode ser finalizada', p_codigo_os;
    END IF;

    UPDATE mecanica.ordem_servico
    SET status_os = 'Finalizado',
        data_fechamento = CURRENT_DATE
    WHERE codigo_os = p_codigo_os;

    INSERT INTO mecanica.garantia_servico
    (id_item, data_inicio, data_fim, km_inicio, km_fim)
    SELECT
        i.id_item,
        CURRENT_DATE,
```



```

CURRENT_DATE + s.garantia_dias,
os.quilometragem,
os.quilometragem + s.garantia_km
FROM mecanica.item_os_servico i
JOIN mecanica.servico s ON s.codigo_servico = i.codigo_servico
JOIN mecanica.ordem_servico os ON os.codigo_os = i.codigo_os
WHERE i.codigo_os = p_codigo_os
AND os.categoria = 'Manutencao_Veicular';
END;
$$;

```

6.4. Segurança de Herança e Views Atualizáveis (vw_clientes)

Devido à adoção da arquitetura de especialização *Table Per Type*, a entidade raiz **Pessoa** armazena simultaneamente os dados civis (nome, endereço, contatos) de clientes, atendentes, mecânicos e da própria gerência. Conceder permissão direta de escrita (INSERT/UPDATE) na tabela **Pessoa** ao perfil de Atendente criaria uma vulnerabilidade grave, permitindo a alteração indevida de dados de outros funcionários.

Para mitigar esse risco sem engessar a operação, foi implementada uma arquitetura de **Views Atualizáveis (Updatable Views)** combinada com gatilhos de interceptação (**INSTEAD OF Triggers**), bloqueando o acesso direto às tabelas base e abstraindo a transação de herança múltipla.

Implementação da View e Triggers:

```

-- =====
-- VIEW: Expõe apenas dados do domínio de Clientes
-- =====
CREATE OR REPLACE VIEW mecanica.vw_clientes AS
SELECT
    p.cpf, p.nome, p.email, p.logradouro, p.numero, p.bairro, p.cep, p.id_cidade,
    c.data_cadastro
FROM mecanica.pessoa p
INNER JOIN mecanica.cliente c ON c.cpf_cliente = p.cpf;

-- =====
-- TRIGGER FUNCTION: Gerencia Inserções na Herança
-- =====
CREATE OR REPLACE FUNCTION mecanica.fn_insert_vw_clientes()
RETURNS TRIGGER SECURITY DEFINER AS $$

```

```

BEGIN
    INSERT INTO mecanica.pessoa (cpf, nome, email, logradouro, numero, bairro, cep, id_
VALUES (NEW.cpf, NEW.nome, NEW.email, NEW.logradouro, NEW.numero, NEW.bairr
ON CONFLICT (cpf) DO UPDATE SET
    nome = EXCLUDED.nome,
    email = EXCLUDED.email,
    logradouro = EXCLUDED.logradouro,
    numero = EXCLUDED.numero,
    bairro = EXCLUDED.bairro,
    cep = EXCLUDED.cep,
    id_cidade = EXCLUDED.id_cidade;

    INSERT INTO mecanica.cliente (cpf_cliente, data_cadastro)
VALUES (NEW.cpf, COALESCE(NEW.data_cadastro, now()))
ON CONFLICT (cpf_cliente) DO NOTHING;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_insert_vw_clientes
    INSTEAD OF INSERT ON mecanica.vw_clientes
    FOR EACH ROW EXECUTE FUNCTION mecanica.fn_insert_vw_clientes();

-- =====
-- TRIGGER FUNCTION: Gerencia Atualizações
-- =====

CREATE OR REPLACE FUNCTION mecanica.fn_update_vw_clientes()
RETURNS TRIGGER SECURITY DEFINER AS $$
BEGIN
    UPDATE mecanica.pessoa
    SET
        nome      = NEW.nome,
        email     = NEW.email,
        logradouro = NEW.logradouro,
        numero    = NEW.numero,
        bairro    = NEW.bairro,
        cep       = NEW.cep,
        id_cidade = NEW.id_cidade
    WHERE cpf = OLD.cpf;

```

```

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_vw_clientes
    INSTEAD OF UPDATE ON mecanica.vw_clientes
    FOR EACH ROW EXECUTE FUNCTION mecanica.fn_update_vw_clientes();

---
```

7. População de Dados

O banco foi populado com um script coeso que respeita os mínimos exigidos e cobre ca

7.1. Contagem de Tuplas por Tabela

Tabela Tuplas Mínimo Tipo
--- --- --- ---
estado 5 5 Principal
cidade 5 5 Principal
pessoa 16 5 Principal
telefones 14 5 Principal
funcionario 10 5 Principal
atendente 5 5 Principal
mecanico 5 5 Principal
cliente 6 5 Principal
veiculo 5 5 Principal
peca 5 5 Principal
servico 7 5 Principal
ordem_servico 10 5 Principal
item_os_servico 13 10 Associativa
ordem_servico_peca 10 10 Associativa
rateio_mecanico_servico 14 10 Associativa
ordem_pagamento 10 10 Fato
garantia_servico 10 10 Fato
solicitacao_compra 10 5 Principal

7.2. Casos de Borda Cobertos

- ****LEFT JOIN:**** `Fernanda Lins` cadastrada como cliente sem veículo e sem nenhuma OS
- ****RIGHT JOIN:**** Serviços `Higienização Interna` e `Polimento Externo` cadastrados no sistema
- ****Rateio dividido:**** Item 3 (Revisão de Freio, OS 2) dividido 50% entre José e Marcos
- ****Pagamentos mistos:**** OS 4 possui dois registros de pagamento parcial com tipos diferentes
- ****Garantia acionada:**** Item 3 (freio da OS 2) com `utilizada = true`, `data\_utilizacao` por 12 meses
- ****OS em garantia:**** OS 9 registra o atendimento de retorno sem gerar pagamento — apenas para diagnóstico
- ****Venda balcão:**** OS 3, 5 e 8 sem placa, sem quilometragem e sem registro em `garantia`
- ****Todos os 5 atendentes e 5 mecânicos**** participam de ao menos uma OS ou item de OS

7.3. Script de Populamento

```

```sql
-- 1. ESTADO
INSERT INTO mecanica.estado (sigla, nome) VALUES
('AL', 'Alagoas'), ('PE', 'Pernambuco'), ('SE', 'Sergipe'),
('BA', 'Bahia'), ('SP', 'São Paulo');

-- 2. CIDADE
INSERT INTO mecanica.cidade (nome, id_estado) VALUES
('Arapiraca', 1), ('Maceió', 1), ('Caruaru', 2),
('Aracaju', 3), ('Salvador', 4);

-- 3. PESSOA (16 tuplas — atendentes, mecânicos e clientes)
INSERT INTO mecanica.pessoa
(cpf, nome, email, logradouro, numero, bairro, cep, id_cidade) VALUES
('111111111111', 'Carlos Augusto', 'carlos.atend@gmail.com', 'Rua XV de Novembro', 100, 'Centro', '51201-000', 1),
('222222222222', 'Mariana Costa', 'mariana.atend@hotmail.com', 'Av. Venturosa', 150, 'Centro', '51201-000', 1),
('333333333333', 'Roberto Almeida', 'roberto.atend@yahoo.com', 'Rua São Francisco', 200, 'Centro', '51201-000', 1),
('10101010101', 'Juliana Torres', 'juliana.atend@gmail.com', 'Rua Nova', 120, 'Centro', '51201-000', 1),
('20202020202', 'Ricardo Mendes', 'ricardo.atend@hotmail.com', 'Av. Brasil', 199, 'Centro', '51201-000', 1),
('444444444444', 'José Silva', 'ze.mecanico@gmail.com', 'Rua Delmiro Gouveia', 110, 'Centro', '51201-000', 1),
('555555555555', 'Lucas Santos', 'lucas.mecanico@outlook.com', 'Av. Deputada Ceci Curvelo', 150, 'Centro', '51201-000', 1),
('666666666666', 'Marcos Oliveira', 'marcos.mecanico@gmail.com', 'Rua Santa Rita', 150, 'Centro', '51201-000', 1),
('777777777777', 'André Souza', 'andre.mecanico@hotmail.com', 'Rua Pedro II', 110, 'Centro', '51201-000', 1),
('30303030303', 'Fábio Júnior', 'fabio.mecanico@yahoo.com', 'Rua do Sol', 155, 'Centro', '51201-000', 1),
('888888888888', 'Arthur Pendragon', 'arthur.cliente@gmail.com', 'Av. Fernandes Lima', 150, 'Centro', '51201-000', 1),
('999999999999', 'Beatriz Silva', 'beatriz.cliente@hotmail.com', 'Rua Agapito Magalhães', 150, 'Centro', '51201-000', 1),
('000000000000', 'Claudio Duarte', 'claudio.cliente@yahoo.com', 'Rua São João', 150, 'Centro', '51201-000', 1),
('12345678901', 'Diana Prince', 'diana.cliente@gmail.com', 'Rua Rui Barbosa', 199, 'Centro', '51201-000', 1),
('98765432109', 'Eduardo Ramos', 'eduardo.cliente@outlook.com', 'Av. Hermes Fontes', 150, 'Centro', '51201-000', 1);

```

```
('10293847560', 'Fernanda Lins', 'fernanda.semOS@gmail.com', 'Rua das Flores',

-- (... o resto está no documento 03_populacao_dados.sql)
```

---

## 8. Consultas e Relatórios (DQL)

### Consulta 1 — Seleção Simples: Catálogo de Serviços com Garantia

Lista todos os serviços disponíveis com seus parâmetros de garantia, ordenados pelo valor.

```
SELECT
 descricao,
 valor,
 tempo_estimado_horas AS horas,
 garantia_dias,
 garantia_km
FROM mecanica.servico
ORDER BY valor;
```

---

### Consulta 2 — INNER JOIN: Veículos e seus Proprietários

Retorna todos os veículos cadastrados com o nome do cliente dono.

```
SELECT
 v.placa,
 v.marca,
 v.modelo,
 v.ano,
 v.cor,
 p.nome AS proprietario
FROM mecanica.veiculo v
INNER JOIN mecanica.cliente c ON c.cpf_cliente = v.cpf_dono
INNER JOIN mecanica.pessoa p ON p.cpf = c.cpf_cliente
ORDER BY p.nome;
```

---

### Consulta 3 — LEFT JOIN: Clientes sem Veículo

Exibe todos os clientes, incluindo os que não possuem nenhum veículo cadastrado.

```
SELECT
 p.nome AS cliente,
 p.email,
 v.placa,
 v.modelo
FROM mecanica.cliente c
INNER JOIN mecanica.pessoa p ON p.cpf = c.cpf_cliente
LEFT JOIN mecanica.veiculo v ON v.cpf_dono = c.cpf_cliente
ORDER BY v.placa NULLS LAST;
```

### Consulta 4 — RIGHT JOIN: Serviços Nunca Executados

Identifica serviços do catálogo que ainda não foram aplicados em nenhuma OS.

```
SELECT
 s.codigo_servico,
 s.descricao,
 s.valor,
 i.codigo_os
FROM mecanica.item_os_servico i
RIGHT JOIN mecanica.servico s ON s.codigo_servico = i.codigo_servico
WHERE i.id_item IS NULL
ORDER BY s.valor DESC;
```

### Consulta 5 — GROUP BY e HAVING: Faturamento por Atendente

Agrupa o total faturado por atendente, exibindo apenas os que ultrapassaram R\$ 300,00 em OS finalizadas.

```
SELECT
 p.nome AS atendente,
 COUNT(os.codigo_os) AS total_os,
 SUM(f.valor_total_fatura) AS faturamento_total
FROM mecanica.ordem_servico os
```

```
INNER JOIN mecanica.atendente a ON a.cpf_atendente = os.cpf_atendente
INNER JOIN mecanica.pessoa p ON p.cpf = a.cpf_atendente
INNER JOIN mecanica.vw_faturamento_os f ON f.codigo_os = os.codigo_os
WHERE os.status_os = 'Finalizado'
GROUP BY p.nome
HAVING SUM(f.valor_total_fatura) > 300
ORDER BY faturamento_total DESC;
```

---

## Consulta 6 — Subconsulta Não Correlacionada: OS acima da Média

Lista todas as OS cujo faturamento supera a média geral de todas as OS.

```
SELECT
 f.codigo_os,
 f.descricao,
 f.status_os,
 f.valor_total_fatura
FROM mecanica.vw_faturamento_os f
WHERE f.valor_total_fatura > (
 SELECT AVG(valor_total_fatura)
 FROM mecanica.vw_faturamento_os
)
ORDER BY f.valor_total_fatura DESC;
```

---

## Consulta 7 — Subconsulta Correlacionada: Mecânicos com mais de um Serviço

Retorna mecânicos que executaram serviços em mais de uma OS distinta.

```
SELECT
 p.nome AS mecanico,
 f.salario_base,
 (
 SELECT COUNT(DISTINCT i.codigo_os)
 FROM mecanica.rateio_mecanico_servico r
 INNER JOIN mecanica.item_os_servico i ON i.id_item = r.id_item
 WHERE r.cpf_mecanico = m.cpf_mecanico
)
```

```

) AS total_os_atendidas
FROM mecanica.mecanico m
INNER JOIN mecanica.funcionario f ON f.cpf_funcionario = m.cpf_mecanico
INNER JOIN mecanica.pessoa p ON p.cpf = m.cpf_mecanico
WHERE (
 SELECT COUNT(DISTINCT i.codigo_os)
 FROM mecanica.rateio_mecanico_servico r
 INNER JOIN mecanica.item_os_servico i ON i.id_item = r.id_item
 WHERE r.cpf_mecanico = m.cpf_mecanico
) > 1
ORDER BY total_os_atendidas DESC;

```

---

## Consulta 8 — Operador de Conjunto INTERSECT: Clientes com Veículo e OS Finalizada

Retorna apenas os clientes que possuem veículo cadastrado E têm ao menos uma OS finalizada — interseção real de dois conjuntos.

```

SELECT cpf_cliente AS cpf
FROM mecanica.cliente c
INNER JOIN mecanica.veiculo v ON v.cpf_dono = c.cpf_cliente

INTERSECT

SELECT c.cpf_cliente AS cpf
FROM mecanica.ordem_servico os
INNER JOIN mecanica.veiculo v ON v.placa = os.placa_veiculo
INNER JOIN mecanica.cliente c ON c.cpf_cliente = v.cpf_dono
WHERE os.status_os = 'Finalizado';

```

---

## Consulta 9 — Operador de Conjunto EXCEPT: Peças Nunca Solicitadas para Compra

Identifica peças do estoque que nunca geraram uma solicitação de compra.

```

SELECT codigo_peca, nome
FROM mecanica.peca

```



```
EXCEPT
```

```
SELECT p.codigo_peca, p.nome
FROM mecanica.peca p
INNER JOIN mecanica.solicitacao_compra sc ON sc.peca_solicitada = p.codigo_peca;
```

---

## Consulta 10 — View + JOIN: Garantias Ativas por Veículo

Usa a view de faturamento combinada com os dados de garantia para exibir o histórico completo de cobertura ativa por veículo.

```
SELECT
 v.placa,
 v.modelo,
 p.nome AS proprietario,
 s.descricao AS servico,
 g.data_inicio,
 g.data_fim,
 g.km_inicio,
 g.km_fim,
 g.utilizada,
 f.valor_total_fatura AS valor_os
FROM mecanica.garantia_servico g
INNER JOIN mecanica.item_os_servico i ON i.id_item = g.id_item
INNER JOIN mecanica.servico s ON s.codigo_servico= i.codigo_servico
INNER JOIN mecanica.ordem_servico os ON os.codigo_os = i.codigo_os
INNER JOIN mecanica.veiculo v ON v.placa = os.placa_veiculo
INNER JOIN mecanica.cliente c ON c.cpf_cliente = v.cpf_dono
INNER JOIN mecanica.pessoa p ON p.cpf = c.cpf_cliente
INNER JOIN mecanica.vw_faturamento_os f ON f.codigo_os = os.codigo_os
WHERE g.utilizada = false
 AND g.data_fim >= CURRENT_DATE
ORDER BY g.data_fim;
```

---

## 9. Segurança e Controle de Acesso (RBAC)

Para garantir que cada usuário tenha acesso apenas aos dados pertinentes à sua função e em conformidade com as diretrizes de privacidade, foram criados três papéis fundamentais para o domínio do projeto:

- **Gerente:** Possui controle total administrativo sobre todas as tabelas e sequências do esquema, representando o nível mais alto de privilégio.
- **Atendente:** Responsável pela recepção e cadastro inicial, possuindo permissões operacionais de leitura, inserção e atualização em tabelas do dia a dia.
- **Mecânico:** Perfil puramente técnico e altamente restrito, focado na execução dos serviços e atualização de ordens de trabalho em andamento.

Concedeu-se o privilégio `USAGE` no esquema `mechanica` para todas as *roles*. Sem este comando, os usuários não teriam autorização sequer para acessar o esquema, gerando falhas de acesso imediato.

**Matriz de Privilégios por Papel**

Tabela / Objeto	role_gerente	role_atendente	role_mecanico
Todas as Tabelas e Sequences	ALL PRIVILEGES	Sem acesso direto total	Sem acesso direto total
Tabelas Operacionais (Veículo, Pagamento, etc.)	ALL PRIVILEGES	SELECT, INSERT, UPDATE	Nenhum acesso direto
Tabela Ordem de Serviço (OS)	ALL PRIVILEGES	SELECT, INSERT, UPDATE	UPDATE restrito (status, km, desc)
Tabela Pessoa e Cliente	ALL PRIVILEGES	Nenhum acesso direto	Nenhum acesso direto
Visão de Clientes ( vw_clientes )	ALL PRIVILEGES	SELECT, INSERT, UPDATE	Nenhum acesso direto
Tabela de Estoque (Peça)	ALL PRIVILEGES	SELECT, INSERT, UPDATE	SELECT
Tabela Funcionario	ALL PRIVILEGES	SELECT restrito à coluna CPF	Nenhum acesso direto
Catálogos e Apoio (Serviço, Estado, Cidade)	ALL PRIVILEGES	SELECT	SELECT (apenas Serviço e Itens)
Visão do Painel ( vw_painel_mecanico )	ALL PRIVILEGES	Nenhum acesso necessário	SELECT

---

## Script de Concessão de Privilégios (GRANTS)

Abaixo estão os comandos DCL (*Data Control Language*) que materializam a matriz de acessos detalhada acima, incluindo a limitação do Atendente à `vw_clientes` para proteção da hierarquia de funcionários:

```
-- =====
-- 1. CRIAÇÃO DOS PAPÉIS E ACESSO AO SCHEMA
-- =====

CREATE ROLE role_gerente;
CREATE ROLE role_atendente;
CREATE ROLE role_mecanico;

-- Concedendo acesso básico ao schema para todas as roles
GRANT USAGE ON SCHEMA mecanica TO role_gerente, role_atendente, role_mecanico;

-- =====
-- 2. PRIVILÉGIOS: GERENTE
-- =====

GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA mecanica TO role_gerente;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA mecanica TO role_gerente;

-- =====
-- 3. PRIVILÉGIOS: ATENDENTE
-- =====

-- Permissão de leitura em tabelas de catálogo/apoio
GRANT SELECT ON
 mecanica.estado, mecanica.cidade, mecanica.servico,
 mecanica.mecanico, mecanica.atendente
TO role_atendente;

-- Leitura restrita (apenas CPF) na tabela de funcionários
GRANT SELECT (cpf_funcionario) ON mecanica.funcionario TO role_atendente;

-- O Atendente precisa poder LER as tabelas pessoa e cliente (devido a FKs/JOINS nas co
GRANT SELECT ON mecanica.pessoa, mecanica.cliente TO role_atendente;

-- Permissão de manipulação nas tabelas operacionais do dia a dia e na nova View de CI
```

```

GRANT SELECT, INSERT, UPDATE ON
 mecanica.vw_clientes,
 mecanica.telefones,
 mecanica.veiculo,
 mecanica.ordem_servico,
 mecanica.ordem_pagamento,
 mecanica.solicitacao_compra,
 mecanica.ordem_servico_peca,
 mecanica.item_os_servico,
 mecanica.garantia_servico,
 mecanica.peca
TO role_atendente;

-- Permissão nas sequences para gerar novos IDs (PKs) nos INSERTS
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA mecanica TO role_atendente;

-- =====
-- 4. PRIVILÉGIOS: MECÂNICO
-- =====
-- Permissão exclusiva de LEITURA para consultar catálogos e itens da OS
GRANT SELECT ON
 mecanica.peca, mecanica.servico, mecanica.item_os_servico,
 mecanica.ordem_servico_peca, mecanica.garantia_servico,
 mecanica.rateio_mecanico_servico
TO role_mecanico;

-- Leitura da view de painel com os dados filtrados do serviço atual
GRANT SELECT ON mecanica.vw_painel_mecanico TO role_mecanico;

-- Atualização restrita a atributos técnicos do andamento do trabalho
GRANT UPDATE (status_os, quilometragem, descricao) ON mecanica.ordem_servico TO

```

## 10. Criptografia de Dados Sensíveis

Para elevar o nível de maturidade de segurança do banco de dados e garantir a confidencialidade das credenciais de acesso do sistema da oficina, foi implementada uma camada de criptografia nativa:

- **Ativação da Extensão pgcrypto** : Através do comando `CREATE EXTENSION` , foi habilitado o módulo oficial de funções criptográficas do PostgreSQL. Isso permite o processamento seguro de dados sensíveis diretamente no motor de banco de dados, blindando o ecossistema.
  - **Hashing com Algoritmo Blowfish (Bcrypt)**: A coluna `senha_acesso` foi adicionada à tabela `mecanica.funcionario` e povoada utilizando a função `crypt` combinada com o gerador de sal aleatório `gen_salt('bf')` . O uso do algoritmo Blowfish (padrão do Bcrypt) garante que cada senha receba um *salt* único e exclusivo por linha. Isso significa que mesmo se dois funcionários possuírem a mesma senha textual, os *hashes* armazenados no disco serão totalmente diferentes, neutralizando completamente ataques de engenharia reversa via tabelas arco-íris (*Rainbow Tables*).
  - **Hardening de Restrição**: A alteração final da coluna para `SET NOT NULL` aplica uma diretiva de endurecimento (*hardening*) de segurança, garantindo normativamente que nenhum funcionário possa existir na base de dados sem credenciais de autenticação devidamente cadastradas e criptografadas.
- 

## 11. Otimização de Desempenho via Indexação Estratégica

Visando garantir a escalabilidade física do banco de dados e evitar varreduras sequenciais custosas (*Sequential Scans*) à medida que a oficina cresce, foram criados três índices estruturados em árvore binária (B-Tree):

- **idx\_os\_status (Otimização da Visão)**: Indexa a coluna `status_os` na tabela de ordens de serviço. Este índice possui um papel estratégico crucial, pois dá suporte imediato ao filtro horizontal (cláusula `WHERE` ) utilizado na visão `vw_painel_mecanico` . Ele permite que o banco localize instantaneamente apenas os registros ativos, acelerando o tempo de resposta da tela operacional dos mecânicos.
  - **idx\_veiculo\_dono (Otimização de Junções)**: Indexa a chave estrangeira `cpf_dono` na tabela `veiculo` . Ele atua diretamente na aceleração das operações de acoplamento ( `INNER JOIN` ) executadas entre as tabelas `veiculo` , `cliente` e `pessoa` , impedindo gargalos de processamento durante a consolidação de relatórios e busca de histórico no balcão.
  - **idx\_os\_data\_abertura (Métricas Temporais)**: Indexa o campo cronológico `data_abertura` das ordens de serviço. Sua presença assegura que consultas gerenciais de auditoria, ordenação por prioridade de chegada e relatórios de produtividade mensal sejam respondidos em tempo submilissegundo pelo sistema de gerenciamento.
-

## 12. Análise Crítica e Conclusão

O desenvolvimento deste projeto exigiu um equilíbrio cuidadoso entre a teoria relacional estrita e a aplicabilidade em um sistema real operando em ambiente produtivo.

**Decisão Arquitetural - Remoção do `valor_total` :** Manter o valor total armazenado fisicamente na tabela `ordem_servico` violaria a 3FN, pois esse valor depende transitivamente dos itens da OS e não da OS em si. A solução via `VIEW` ( `vw_faturamento_os` ) garante consistência permanente sem custo de manutenção ou risco de anomalias de atualização.

**Especialização via Table Per Type e Triggers ( `INSTEAD OF` ):** A hierarquia `Pessoa` -> `Funcionário` -> `Mecânico` / `Atendente` evita colunas nulas estruturais e mantém o domínio coeso. Contudo, na arquitetura de acessos, isso expunha dados civis de funcionários aos atendentes. A solução arquitetural de criar a visão `vw_clientes` acoplada a gatilhos de interceptação ( `Triggers INSTEAD OF` ) atuou de forma magistral: o `Atendente` interage exclusivamente com a View, e o PostgreSQL assume a responsabilidade transacional de distribuir a inserção nas tabelas base. A inclusão da cláusula `ON CONFLICT (Upsert)` resolveu a herança múltipla (ex: funcionário tornando-se cliente) sem gerar quebras de integridade.

**Agregação como solução para o rateio:** Sem a entidade associativa `item_os_servico` , seria impossível vincular a comissão de um mecânico a um serviço específico faturado dentro de uma OS. A agregação transforma o relacionamento em entidade, resolvendo o problema de cardinalidade e rastreabilidade financeira.

**Garantia com lógica temporal:** O desafio operacional de assegurar que as garantias existam apenas para Ordens de Serviço efetivamente finalizadas foi sanado diretamente no banco de dados através da procedure `sp_finalizar_os` .

Por fim, a evolução do sistema incorporando o **Controle de Acesso Baseado em Papéis (RBAC)**, o uso de **Views restritas** para a proteção de dados civis sensíveis (LGPD) e a aplicação de **Criptografia nativa ( `pgcrypto` )** consolidou um ecossistema seguro e blindado contra acessos indevidos. A adoção da **indexação estratégica em B-Tree** fechou o ciclo do projeto, assegurando que o provisionamento destas camadas de segurança e o inevitável inchaço volumétrico do banco de dados não comprometerão a vazão, o tempo de resposta e a resiliência operacional da oficina a longo prazo.